Lab4: Text Classification with Multinomial Naive Bayes

Duration: 100 minutes
Course: Data Mining
Dataset: `20newsgroups` from `sklearn.datasets`

Learning Objectives

By the end of this lab, you should be able to:

- understand the structure of a text classification dataset
- prepare text data for machine learning
- convert raw text into numerical features using bag-of-words and TF-IDF
- train a **Multinomial Naive Bayes** classifier
- evaluate the classifier using appropriate metrics
- interpret model performance and common classification errors

Part 1. Introduction

Background

Text classification is a supervised learning task where the input is text and the output is a predefined category.

Multinomial Naive Bayes is widely used for text classification because:

- it is simple and efficient
- it works well with word frequency features
- it often provides strong baseline performance

Questions to think about

1. Why is text data harder to use directly than numerical data?
2. Why must we transform text into features?
3. Why is Multinomial Naive Bayes especially suitable for document classification?

Part 2. Data Loading and Exploration (20 minutes)

Task 1: Load the dataset

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

Load the 20 Newsgroups training and test sets.

TODO

- Import the required modules
- Load the training and test subsets
- Print the number of documents in each subset
- Print the list of class names

In [1]:
```
# Import libraries
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
    precision_recall_fscore_support
)
from sklearn.model_selection import train_test_split


pd.set_option("display.max_columns", None)
pd.set_option("display.width", 140)

# Load the 20 Newsgroups dataset
# 加载 20 Newsgroups 数据集的训练集和测试集，并自动移除邮件头、页脚和引用内容，只
# 数据集比较大，加载可能需要一些时间。
train_subset = fetch_20newsgroups(data_home = './dataset',subset="train", remove
test_subset = fetch_20newsgroups(data_home='./dataset',subset="test", remove=("h

list(train_subset.target_names)
```

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

Out[1]:
```
['alt.atheism',
 'comp.graphics',
 'comp.os.ms-windows.misc',
 'comp.sys.ibm.pc.hardware',
 'comp.sys.mac.hardware',
 'comp.windows.x',
 'misc.forsale',
 'rec.autos',
 'rec.motorcycles',
 'rec.sport.baseball',
 'rec.sport.hockey',
 'sci.crypt',
 'sci.electronics',
 'sci.med',
 'sci.space',
 'soc.religion.christian',
 'talk.politics.guns',
 'talk.politics.mideast',
 'talk.politics.misc',
 'talk.religion.misc']
```

In [2]:
```
print(train_subset.target.shape)
```

```
(11314,)
```

Task 2: Explore sample documents

Inspect a few training examples.

TODO

- Print 3 example documents
- For each document, print:
  - document index
  - target label index
  - target label name
  - first part of the text

In [3]:
```
# TODO: Display a few example documents from the training set
import random
random_indices = random.sample(range(len(train_subset.data)), 3)


for i in random_indices:
    print(f'Document Index:',i)
    print(f'target label index:',train_subset.target[i])
    print(f'target label name:',train_subset.target_names[train_subset.target[i]
    print('Text content(200 characters only):')
    print(train_subset.data[i][:200])
    print(' ')
```

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

```
Document Index: 2190
target label index: 16
target label name: talk.politics.guns
Text content(200 characters only):
/ iftccu:talk.politics.guns / steiner@jupiter.ca.boeing.com / 12:07 am  Apr 15, 1
993 /

Also, you need to consider our legal system.  Since any of these things
CAN be lethal, you are going to have a h

Document Index: 9492
target label index: 9
target label name: rec.sport.baseball
Text content(200 characters only):
Went to the Dodgers game tonight -- it was cap night.

Astacio pitched ok, but had control trouble all night.
In the first, he walked a batter, balked him to second, then
a single scored the run, wit

Document Index: 3358
target label index: 14
target label name: sci.space
Text content(200 characters only):

 > be the site of major commercial activity. As far as we know it has no
 > materials we can't get cheaper right here on Earth or from asteroids
 > and comets, aside from the semi-mythic He3 that *mi
```

Task 3: Explore class distribution

TODO

- Count how many documents belong to each class in the training set
- Create a table with class names and counts
- Plot a bar chart showing the class distribution

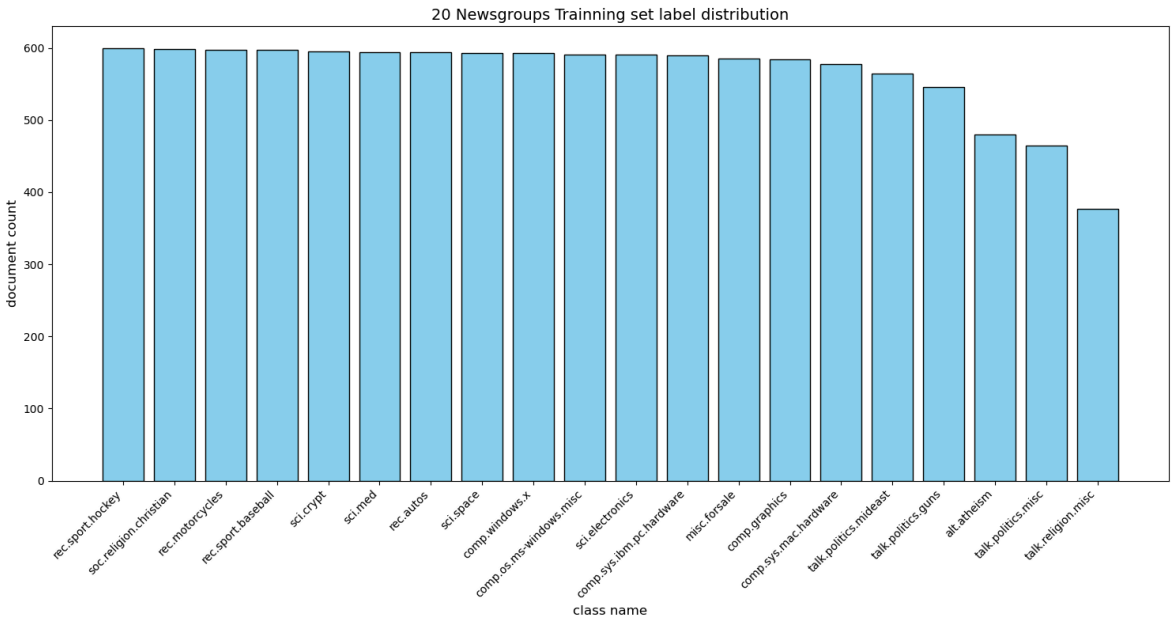
In [4]:

```
# TODO: Analyze and visualize the class distribute
from collections import Counter

class_conuts = Counter(train_subset.target)

df_count = pd.DataFrame(
    {
        'class_index': list(class_conuts.keys()),
        'class_name': [train_subset.target_names[i] for i in class_conuts.keys()
        'document_count': list(class_conuts.values())
    }
)
df_count = df_count.sort_values(by='document_count', ascending=False).reset_inde
print(df_count)
```

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

	class_index	class_name	document_count
0	10	rec.sport.hockey	600
1	15	soc.religion.christian	599
2	8	rec.motorcycles	598
3	9	rec.sport.baseball	597
4	11	sci.crypt	595
5	13	sci.med	594
6	7	rec.autos	594
7	14	sci.space	593
8	5	comp.windows.x	593
9	2	comp.os.ms-windows.misc	591
10	12	sci.electronics	591
11	3	comp.sys.ibm.pc.hardware	590
12	6	misc.forsale	585
13	1	comp.graphics	584
14	4	comp.sys.mac.hardware	578
15	17	talk.politics.mideast	564
16	16	talk.politics.guns	546
17	0	alt.atheism	480
18	18	talk.politics.misc	465
19	19	talk.religion.misc	377

In [5]:
```
#Create a figure to represent the class distribution
plt.figure(figsize=(15, 8))
plt.bar(df_count['class_name'], df_count['document_count'], color='skyblue', edg
plt.xlabel('class name', fontsize=12)
plt.ylabel('document count', fontsize=12)
plt.title('20 Newsgroups Trainning set label distribution', fontsize=14)
plt.xticks(rotation=45, ha='right', fontsize=10)
plt.tight_layout()
plt.show()
```



Student Notes

Write your observations here:

- How many classes are there?

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

- Are the classes balanced?
  - Most a balance, just a few class have distint low amount
- What kinds of topics appear in the dataset?

```
'alt.atheism',<br>
'comp.graphics',<br>
'comp.os.ms-windows.misc',<br>
'comp.sys.ibm.pc.hardware',<br>
'comp.sys.mac.hardware',<br>
'comp.windows.x',<br>
'misc.forsale',<br>
'rec.autos',<br>
'rec.motorcycles',<br>
'rec.sport.baseball',<br>
'rec.sport.hockey',<br>
'sci.crypt',<br>
'sci.electronics',<br>
'sci.med',<br>
'sci.space',<br>
'soc.religion.christian',<br>
'talk.politics.guns',<br>
'talk.politics.mideast',<br>
'talk.politics.misc',<br>
'talk.religion.misc'
```

- Which categories might be easy or difficult to classify?
  - Currently, I don`t know

Part 3. Text Preprocessing and Feature Extraction

Task 1: Convert text into features

Machine learning models cannot use raw text directly.
You will transform the documents into numerical vectors using:

1. **CountVectorizer** (bag-of-words)
2. **TfidfVectorizer** (term frequency-inverse document frequency)

TODO

- Import `CountVectorizer` and `TfidfVectorizer`
- Create a count-based representation of the training and test data
- Create a TF-IDF representation of the training and test data
- Print the shapes of the resulting matrices

In [6]:
```
# TODO: Import CountVectorizer and TfidfVectorizer

t import CountVectorizer, TfidfVectorizer
```

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

In [7]:
```
# TODO: Create bag-of-words features using CountVectorizer
# Suggested settings:
# stop_words="english", max_features=5000
vectorizer = CountVectorizer(stop_words="english", max_features=50000)

X_train_count = vectorizer.fit_transform(train_subset.data)
X_test_count = vectorizer.transform(test_subset.data)

# x= vectorizer.fit_transform(train_subset.data) is equal to:
# vectorizer.fit(train_subset.data)  # Learn the vocabulary from the training da
# x = vectorizer.transform(train_subset.data)  # Transform the training data int
#We create just one varible x, but vectorizer is updated into a new one.
```

In [8]:
```
# TODO: Create TF-IDF features using TfidfVectorizer
# Suggested settings:
# stop_words="english", max_features=5000

tfidf_vectorizer = TfidfVectorizer(stop_words="english", max_features=50000)
X_train_tfidf = tfidf_vectorizer.fit_transform(train_subset.data)
X_test_tfidf = tfidf_vectorizer.transform(test_subset.data)
```

In [9]:
```
# Print out result
print('CountVectorizer:')
print(f'Training set shape: {X_train_count.shape}')
print(f'Testing set shape: {X_test_count.shape}')

print('\nTifidfVectorizer:')
print(f'Training set shape: {X_train_tfidf.shape}')
print(f'Testing set shape: {X_test_tfidf.shape}')
```

```
CountVectorizer:
Training set shape: (11314, 50000)
Testing set shape: (7532, 50000)

TifidfVectorizer:
Training set shape: (11314, 50000)
Testing set shape: (7532, 50000)
```

Task 2: Inspect the vocabulary

TODO

- Print a sample of words from the CountVectorizer vocabulary
- Compare the number of non-zero features in one example document
- Briefly explain why the feature matrix is sparse

In [10]:
```
# TODO: Show part of the learned vocabulary
# TODO: Inspect one document representation

# TASK1: Print a sample of words from the CountVectorizer vocabulary
vocab = vectorizer.vocabulary_
sample_words = list(vocab.keys())[:20]  # Print 20 words
print(sample_words)
```

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

```
#code explanation:
```

```
#We load vectorizer.vocabulary_ to get the whole vocabulary learned by the Count
#now variable vocab contain every english word in training data
```

```
['wondering', 'enlighten', 'car', 'saw', 'day', 'door', 'sports', 'looked', 'lat
e', '60s', 'early', '70s', 'called', 'bricklin', 'doors', 'really', 'small', 'add
ition', 'bumper', 'separate']
```

In [11]:
```
#TASK2: Compare the number of non-zore features in one example document
doc_vector = X_train_count[0]
total_features = X_train_count.shape[1]
print(f'Number of non-zero features in the document vector: {doc_vector.nnz}')
print(f'Total number of features in the vectorizer: {total_features}')
```

```
Number of non-zero features in the document vector: 33
Total number of features in the vectorizer: 50000
```

Why the feature matrix is sparse

The feature matrix is sparse because each docmuent contains only a small subset of all possible words in the vocabulary. Since he vocavulary can include thousands of uniques word, most entries in the document-term matrix are zore. This results in a sprse representation where only a few elements per document have non-zore values.

Student Notes

Answer the following:

1. What is a document-term matrix?
2. What is the difference between CountVectorizer and TfidfVectorizer?
3. Why do we remove stop words?
4. Why is `max_features` useful?

1. A document-term matrix is a numerical representation of a text dataset where each row corresponds to a document and each column corresponds to a unique term(word) from the vocabulary. The values in the matrix represent the frequency or importance(e.g., TF-IDF score) of each term in each document.
2. CountVectorizer converts text into a matrix of raw word counts, where each value represents how many times a wrod appears in a document. In contrast, TfidfVectorizer computes the TF-IDF score, which reflects not only the frequency of a word in a document but also how important the word is across the entire corpus. TF-IDF reduces the influence of common words and emphasized more informative terms.
3. Stop words are common words such as "the", "is", and "and" that appear frequently but carry little meaningful information. Removing them helps reduce noise in the data, decreases dimensionality, and improves model performance by focusing on more informative words.
4. The max_features parameter limits the number of features (i.e., vocabulary [illegible] helps reduce computational cost, prevents

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

overfitting, and removes less important words by keeping only the most frequent or relevant terms.

Part 4. Model Training and Evaluation

Task 1: Train a Multinomial Naive Bayes classifier

You will train two models:

1. Multinomial Naive Bayes with CountVectorizer features
2. Multinomial Naive Bayes with TF-IDF features

TODO

- Import `MultinomialNB`
- Train a model on count features
- Predict labels for the test set
- Compute the test accuracy

In [12]:
```
# TODO: Import MultinomialNB and accuracy_score

from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score
```

In [13]:
```
# TODO: Train Multinomial Naive Bayes on count features
# TODO: Predict the test labels
# TODO: Print the accuracy

model_Multi = MultinomialNB()
model_Multi.fit(X_train_count, train_subset.target)

y_pred = model_Multi.predict(X_test_count)

accuracy = accuracy_score(test_subset.target, y_pred)

print(f'Accuracy of MultinomialNB with CountVectorizer: {accuracy:.4f}')
```

```
Accuracy of MultinomialNB with CountVectorizer: 0.6494
```

Task 2: Evaluate the count-based model

TODO

- Import and print a classification report
- Include precision, recall, and F1-score

In [14]:
```
# TODO: Evaluate the count-based model with classification_report

from sklearn.metrics import classification_report

print('Classification Report(CounterVectorizer):')
```

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

```
oset.target, y_pred, target_names=test_subset
```

```
Classification Report(CounterVectorizer):
                          precision    recall  f1-score   support

             alt.atheism       0.59      0.42      0.49       319
           comp.graphics       0.57      0.72      0.64       389
 comp.os.ms-windows.misc       0.25      0.00      0.01       394
comp.sys.ibm.pc.hardware       0.51      0.73      0.60       392
   comp.sys.mac.hardware       0.70      0.63      0.66       385
          comp.windows.x       0.63      0.77      0.69       395
            misc.forsale       0.85      0.72      0.78       390
               rec.autos       0.79      0.71      0.75       396
         rec.motorcycles       0.87      0.68      0.77       398
      rec.sport.baseball       0.92      0.77      0.84       397
        rec.sport.hockey       0.58      0.88      0.70       399
               sci.crypt       0.63      0.76      0.69       396
         sci.electronics       0.68      0.51      0.58       393
                 sci.med       0.83      0.79      0.81       396
               sci.space       0.75      0.74      0.74       394
  soc.religion.christian       0.49      0.89      0.63       398
      talk.politics.guns       0.57      0.66      0.61       364
   talk.politics.mideast       0.71      0.77      0.74       376
      talk.politics.misc       0.43      0.45      0.44       310
      talk.religion.misc       0.38      0.08      0.13       251

               accuracy                           0.65      7532
              macro avg       0.64      0.63      0.61      7532
           weighted avg       0.64      0.65      0.63      7532
```

Task 3: Train and evaluate the TF-IDF model

TODO

- Train Multinomial Naive Bayes on TF-IDF features
- Predict the test labels
- Print the accuracy
- Print the classification report

In [15]:
```
# TODO: Train and evaluate the TF-IDF model

tfidf_model = MultinomialNB()
tfidf_model.fit(X_train_tfidf, train_subset.target)

y_pred_tfidf = tfidf_model.predict(X_test_tfidf)
accuracy_tfidf = accuracy_score(test_subset.target, y_pred_tfidf)
print(f'Accuracy of MultinomialNB with TfidfVectorizer: {accuracy_tfidf:.4f}')
```

```
Accuracy of MultinomialNB with TfidfVectorizer: 0.6804
```

In [16]:
```
print('Classification Report(TfidfVectorizer):')
print(classification_report(test_subset.target, y_pred_tfidf, target_names=test_
```

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

```
Classification Report(TfidfVectorizer):
                          precision    recall  f1-score   support

             alt.atheism       0.77      0.20      0.31       319
           comp.graphics       0.66      0.69      0.67       389
 comp.os.ms-windows.misc       0.66      0.59      0.62       394
comp.sys.ibm.pc.hardware       0.60      0.74      0.67       392
   comp.sys.mac.hardware       0.77      0.68      0.72       385
          comp.windows.x       0.81      0.77      0.79       395
            misc.forsale       0.78      0.77      0.77       390
               rec.autos       0.82      0.73      0.78       396
         rec.motorcycles       0.85      0.75      0.79       398
      rec.sport.baseball       0.91      0.80      0.85       397
        rec.sport.hockey       0.57      0.93      0.71       399
               sci.crypt       0.61      0.79      0.69       396
         sci.electronics       0.71      0.52      0.60       393
                 sci.med       0.88      0.76      0.82       396
               sci.space       0.76      0.74      0.75       394
  soc.religion.christian       0.38      0.92      0.54       398
      talk.politics.guns       0.57      0.72      0.64       364
   talk.politics.mideast       0.82      0.79      0.80       376
      talk.politics.misc       0.89      0.30      0.45       310
      talk.religion.misc       1.00      0.01      0.02       251

               accuracy                           0.68      7532
              macro avg       0.74      0.66      0.65      7532
           weighted avg       0.74      0.68      0.67      7532
```

Task 4: Compare the two models

TODO

- Create a summary table with:
  - model name
  - accuracy
  - weighted precision
  - weighted recall
  - weighted F1-score
- Decide which model performs better

In [17]:

```python
# TODO: Create a performance comparison table for the two models

from sklearn.metrics import accuracy_score, precision_recall_fscore_support
import pandas as pd

results = []

# ===== CountVectorizer 模型 =====
precision, recall, f1, _ = precision_recall_fscore_support(
    test_subset.target,
    y_pred,
    average='weighted'
)
```

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

```
    "Model": "CountVectorizer + MultinomialNB",
    "Accuracy": accuracy_score(test_subset.target, y_pred),
    "Precision": precision,
    "Recall": recall,
    "F1-score": f1
})


# ===== TF-IDF 模型 =====
precision, recall, f1, _ = precision_recall_fscore_support(
    test_subset.target,
    y_pred_tfidf,
    average='weighted'
)

results.append({
    "Model": "TfidfVectorizer + MultinomialNB",
    "Accuracy": accuracy_score(test_subset.target, y_pred_tfidf),
    "Precision": precision,
    "Recall": recall,
    "F1-score": f1
})


# 转成 DataFrame
df_results = pd.DataFrame(results)

print(df_results)
```

```
                          Model  Accuracy  Precision    Recall  F1-score
0  CountVectorizer + MultinomialNB  0.649363   0.643165  0.649363  0.626898
1  TfidfVectorizer + MultinomialNB  0.680430   0.735337  0.680430  0.667143
```

Task 5: Plot a confusion matrix

TODO

- Import confusion matrix tools
- Plot the confusion matrix for the TF-IDF model
- Identify categories that are commonly confused

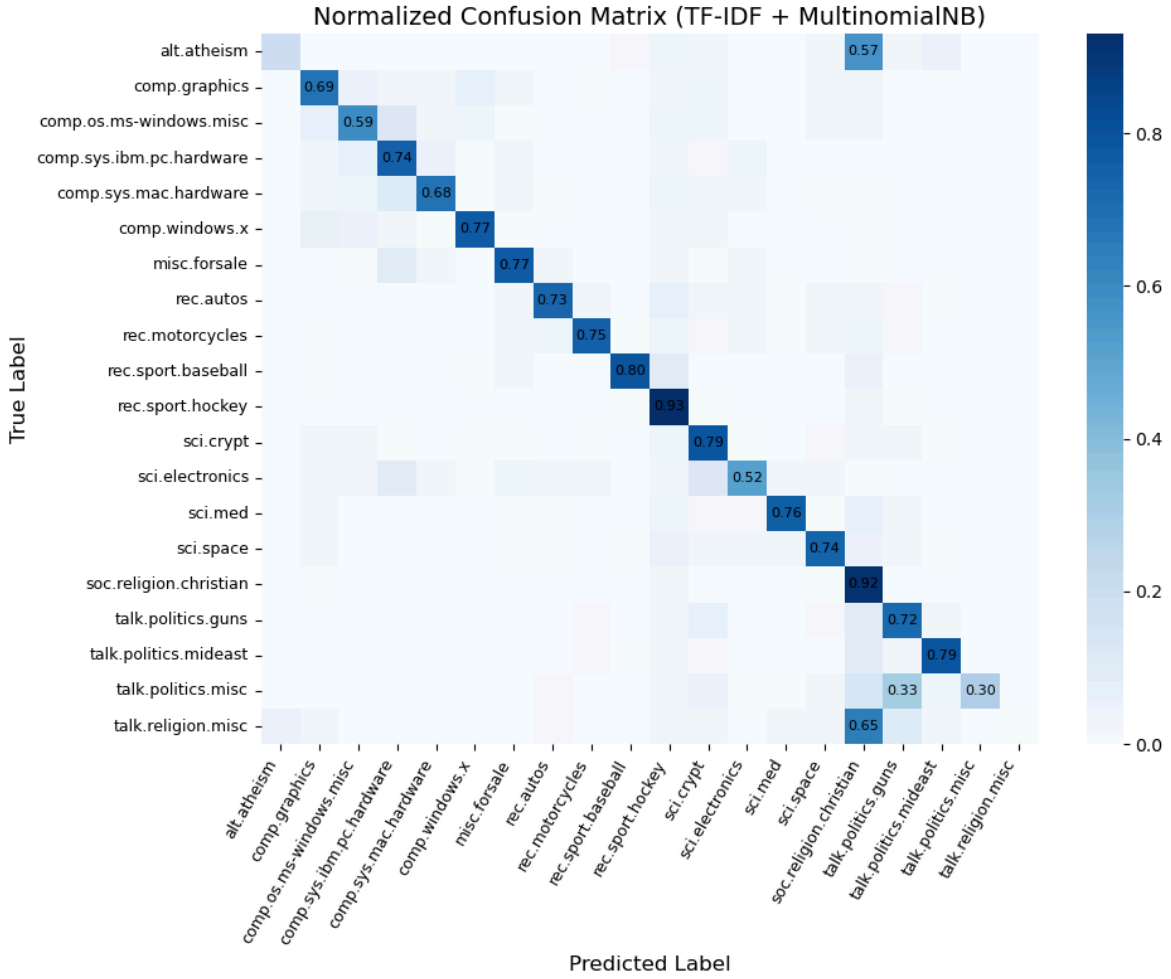
In [21]:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
import seaborn as sns

# 计算混淆矩阵
cm = confusion_matrix(test_subset.target, y_pred_tfidf)

# 归一化（按行）
cm_norm = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]

# 设置画布
plt.figure(figsize=(10, 8))

# 使用seaborn画 heatmap
```

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

```
    cm_norm,
```

```
    cmap="Blues",
    xticklabels=test_subset.target_names,
    yticklabels=test_subset.target_names,
    annot=False,
    fmt=".2f",
    cbar=True
)

# 只标注较大的值（提高可读性）
threshold = 0.2
for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        if cm_norm[i, j] > threshold:
            plt.text(j + 0.5, i + 0.5,
                     f"{cm_norm[i, j]:.2f}",
                     ha="center", va="center",
                     color="black", fontsize=8)

# 美化
plt.xticks(rotation=60, ha='right', fontsize=9)
plt.yticks(rotation=0, fontsize=9)

plt.xlabel("Predicted Label", fontsize=12)
plt.ylabel("True Label", fontsize=12)
plt.title("Normalized Confusion Matrix (TF-IDF + MultinomialNB)", fontsize=14)

plt.tight_layout()
plt.show()
```



Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

Student Notes

Discuss:

- Which feature representation works better?
- Which categories are easiest to classify?
- Which categories are most often confused?
- Why do some categories overlap?

1. The TF-IDF feature representation performs better than the CountVectorizer representation. It achieves higher accuracy, precision, recall, and F1-score. This is because TF-IDF not only considers term frequency but also reduces the impact of commonly occurring words, allowing the model to focus on more informative and discriminative features.

2. Categories with highly distinctive vocabulary are the easiest to classify. For example, topics such as sci.space or rec.sport.hockey often contain domain-specific terms that make them easier for the model to identify correctly.

3. Categories within similar domains are most often confused. For instance, computer-related categories such as comp.graphics and comp.os.ms-windows.misc, or sports-related categories such as rec.sport.baseball and rec.sport.hockey, are frequently misclassified due to their similar vocabulary and context.

4. Some categories overlap because they share similar vocabulary and semantic context. Since bag-of-words models like CountVectorizer and TF-IDF ignore word order and deeper semantic meaning, they cannot fully distinguish between topics that use similar terms. As a result, documents from related categories may appear similar in the feature space, leading to misclassification.

Part 5. Analysis and Conclusion

Task 1: Inspect important words

The model learns probabilities for words in each class.

TODO

- Select one class
- Extract the top 20 words with the highest log probabilities
- Display them in a table
- Explain why these words are relevant to the class

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

In [22]:
```
# TODO: Inspect the top words for one class in the TF-IDF model

class_index = 0
class_name = test_subset.target_names[class_index]

feature_names = tfidf_vectorizer.get_feature_names_out()
log_probs = tfidf_model.feature_log_prob_[class_index]

top_idices = np.argsort(log_probs)[-20:][::-1]

top_words =feature_names[top_idices]
top_scores = log_probs[top_idices]

df_top_words = pd.DataFrame({
    "Word": top_words,
    "Log Probability": top_scores
})

print(f'Top words for class:{class_name}')
print(df_top_words)
```

```
Top words for class:alt.atheism
         Word  Log Probability
0         god        -8.020630
1      people        -8.359588
2         don        -8.390649
3       think        -8.462518
4     atheism        -8.481503
5    religion        -8.530752
6        just        -8.609236
7         say        -8.622903
8    atheists        -8.714992
9       islam        -8.718742
10       does        -8.795917
11      bible        -8.857703
12   morality        -8.903843
13  objective        -8.905136
14       know        -8.920412
15       said        -8.930525
16      moral        -8.953299
17    believe        -8.956433
18      bobby        -8.978940
19    islamic        -8.998783
```

Task 2: Error analysis

TODO

- Find a few misclassified test examples
- Print:
  - the true label
  - the predicted label
  - part of the text
- Explain possible reasons for the errors

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

In [23]:
```
# TODO: Display a few misclassified examples

misclassified_indices = (y_pred_tfidf != test_subset.target)

#We take just front 5 error samples
error_indices = np.where(misclassified_indices)[0][:5]

for i in error_indices:
    print("="*80)
    print("True label:", test_subset.target_names[test_subset.target[i]])
    print("Predicted label:", test_subset.target_names[y_pred_tfidf[i]])
    print("Text snippet:")
    print(test_subset.data[i][:300])  # 只打印前300字符
    print()
```

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

```
================================================================================
True label: comp.windows.x
Predicted label: comp.graphics
Text snippet:
I'm not familiar at all with the format of these "X-Face:" thingies, but
after seeing them in some folks' headers, I've *got* to *see* them (and
maybe make one of my own)!

I've got "dpg-view" on my Linux box (which displays "uncompressed X-Faces")
and I've managed to compile [un]compface too... but

================================================================================
True label: alt.atheism
Predicted label: soc.religion.christian
Text snippet:

In a word, yes.


================================================================================
True label: talk.religion.misc
Predicted label: alt.atheism
Text snippet:

I've just spent two solid months arguing that no such thing as an
objective moral system exists.

================================================================================
True label: soc.religion.christian
Predicted label: comp.os.ms-windows.misc
Text snippet:
A friend of mine managed to get a copy of a computerised Greek and Hebrew
Lexicon called "The Word Perfect" (That is not the word processing
package WordPerfect). However, some one wiped out the EXE file, and she
has not been able to restore it. There are no distributors of the package in
South

================================================================================
True label: alt.atheism
Predicted label: soc.religion.christian
Text snippet:

Probably because it IS rape.


So nothing.  It may work for some, but not for others: it doesn't give any
insight into an overall God or overall truth of a religion- it would seem to be
dependent solely on the individual, as well as individually-created.  And since
Christians have failed to show us
```

Task 3: Final conclusion

Write a short conclusion that includes:

- how the text was transformed into features
- how Multinomial Naive Bayes was used

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

F performed better

- what the main limitations of the model are
- how you could improve the classifier in future work

In this project, raw text data was transformed into numerical features using both CountVectorizer and TF-IDF representations, which convert documents into a document-term matrix based on word frequencies and term importance. A Multinomial Naive Bayes classifier was then trained on these features to perform text classification by learning the probability distribution of words for each category.

Experimental results show that the TF-IDF representation outperforms the CountVectorizer approach, achieving higher accuracy and better overall performance. This is because TF-IDF reduces the impact of common words and emphasizes more informative terms.

However, the model has several limitations. The bag-of-words representation ignores word order and semantic context, and the Naive Bayes model assumes feature independence, which is often unrealistic. These limitations can lead to confusion between categories with similar vocabulary.

In future work, the classifier could be improved by using more advanced feature representations such as word embeddings, applying models that capture context (e.g., deep learning approaches), or tuning hyperparameters to optimize performance.

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js